

CURSO TÉCNICO SUPERIOR PROFISSIONAL DE PROGRAMAÇÃO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO DE APLICAÇÕES

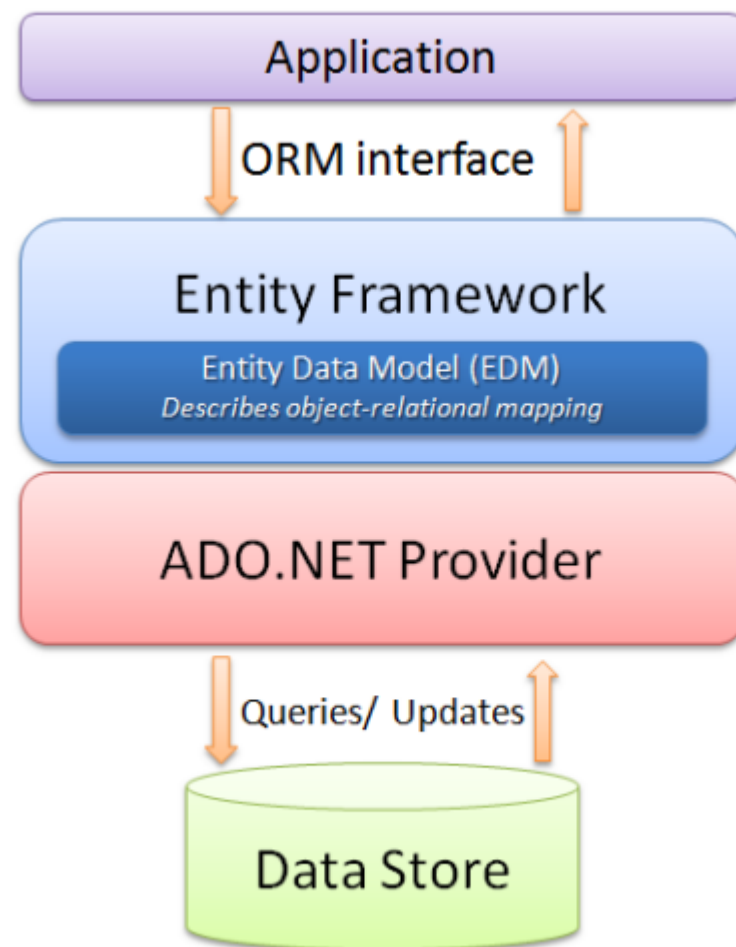
ENTITY FRAMEWORK

ENTITY FRAMEWORK

- É uma *framework* ORM – *Object-Relational Mapping* - para .NET que:
 - Possibilita o mapeamento de tabelas da base de dados em classes de um projeto.
 - Cria automaticamente a camada de acesso a dados.
- Esta *framework* permite ainda:
 - Integrar a tecnologia ADO.NET (tecnologia de acesso a dados para .NET)
 - Utilizar objetos para lidar com bases de dados relacionais
 - Otimizar o tempo de desenvolvimento
 - Libertar o programador para outras tarefas
 - Utilizar a tecnologia LINQ para efetuar consultas

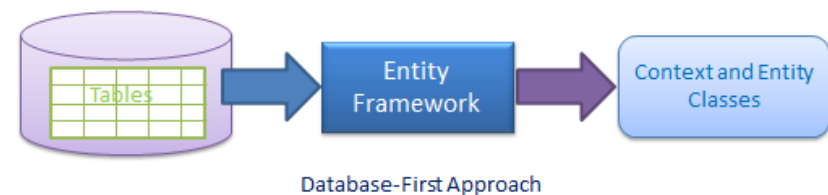
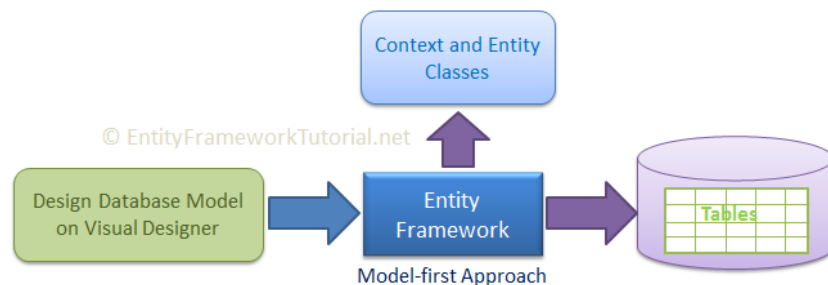
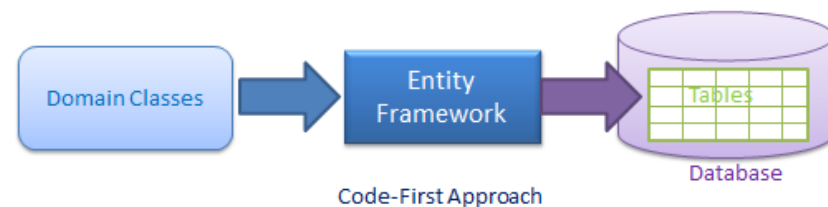
COMO FUNCIONA

Trata-se de uma camada de software intermédia (*middleware*) que trata da manipulação de dados entre uma base de dados e uma aplicação em .NET

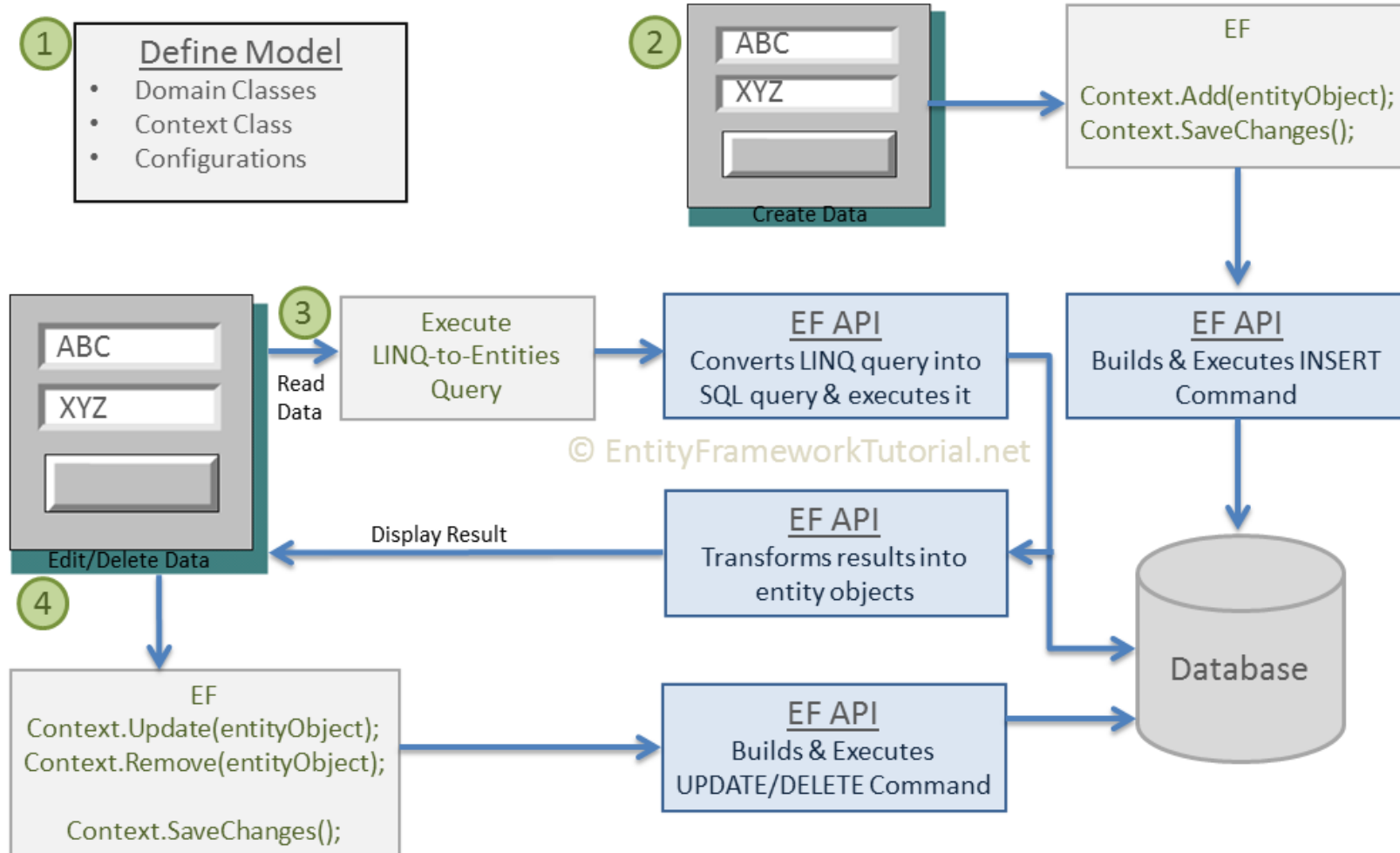


TIPOS DE ABORDAGENS

- **Código primeiro (Code First)** – gerar a base de dados ou/e o modelo a partir de código previamente escrito pelo programador
- **Modelo Primeiro (Model First)** – gerar a base de dados a partir de um modelo previamente definido pelo programador
- **Base de Dados Primeiro (Database First)** – gerar o modelo a partir de uma base de dados existente

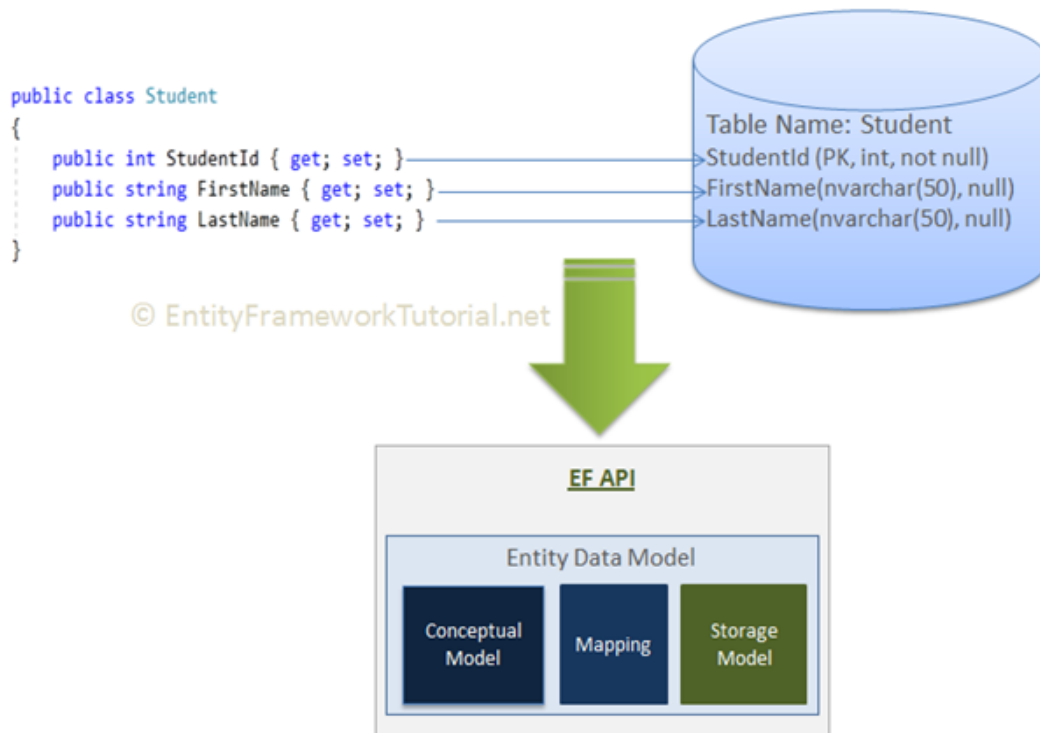


WORKFLOW

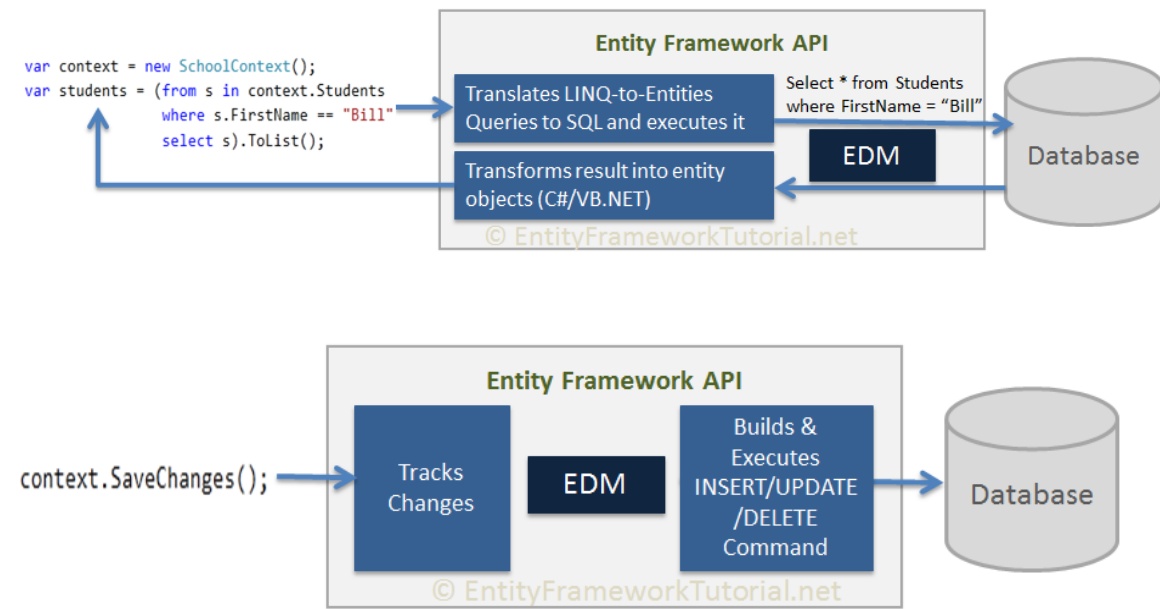


MODELO DE DADOS (EDM)

- O primeiro passo da API EF é criar o Modelo de dados (EDM - *Entity Data Model*)



- Depois é possível efetuar consultas e operações CRUD (Create, Read, Update, Delete)



CONTEXTCLASS

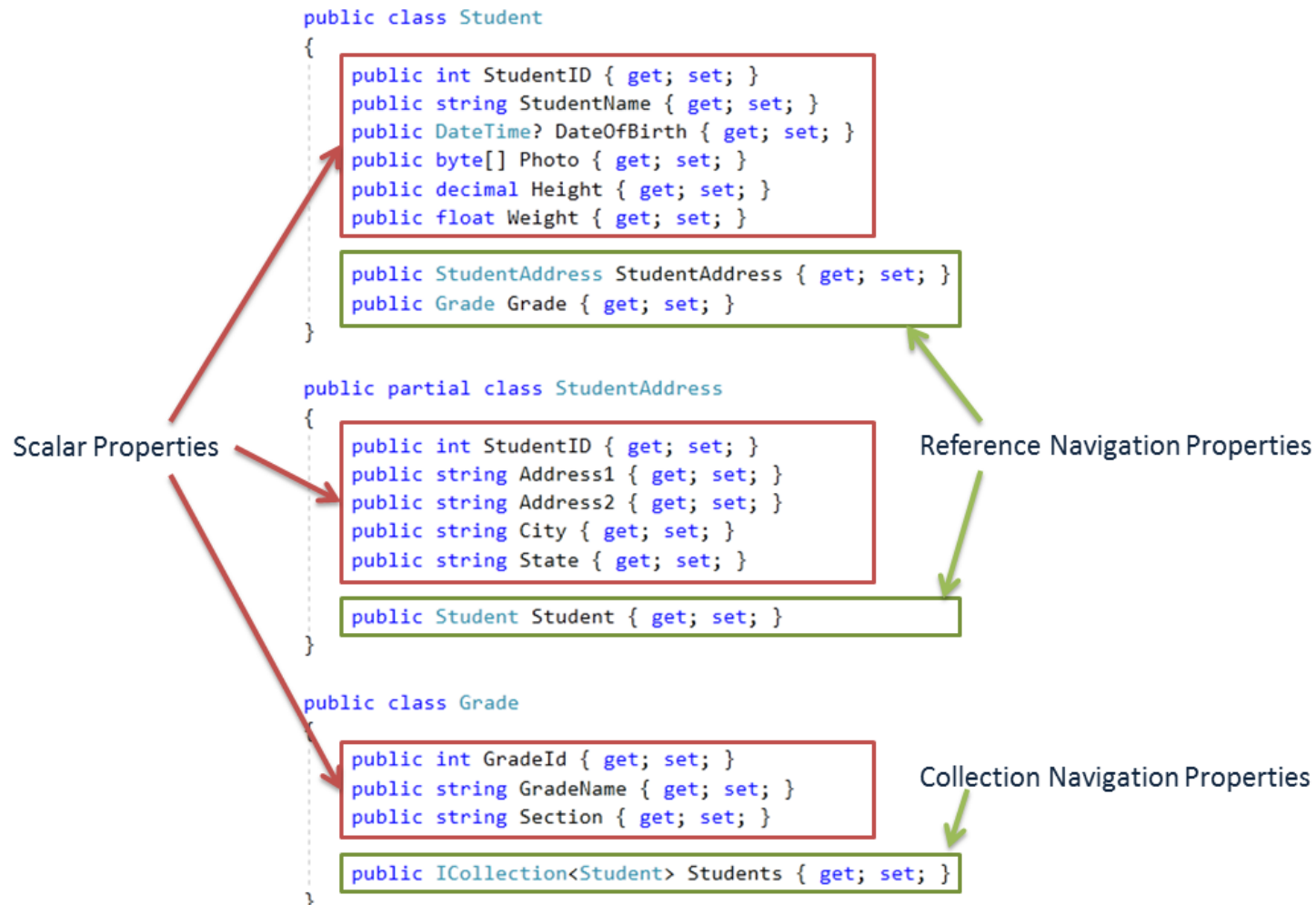
- *ContextClass* é uma classe que herda *DbContext*, e representa a sessão de interligação com a base de dados
- Inclui as estruturas dinâmicas que permitem uma “sincronia” entre os dados na aplicação e a base de dados
- É utilizada para realizar consultas e guardar dados na Base de dados relacionada

```
public class SchoolContext : DbContext {  
    public SchoolContext() { }  
  
    public DbSet<Student> Students { get; set; }  
    public DbSet<StudentAddress> StudentAddresses { get; set; }  
    public DbSet<Grade> Grades { get; set; }  
}
```

ENTITIES E SCALARS

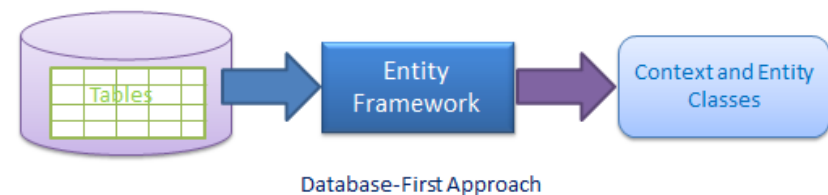
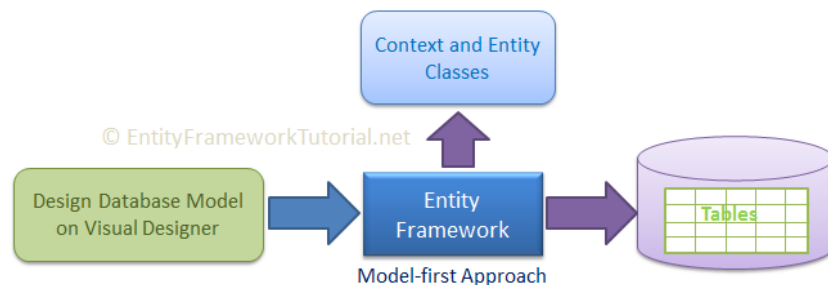
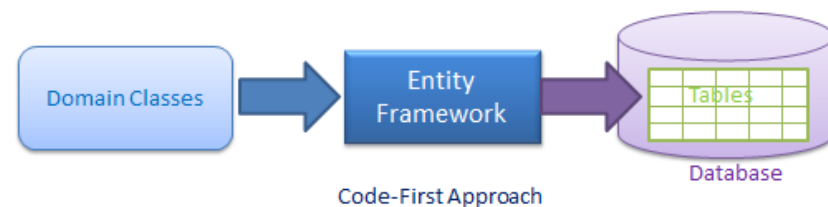
Entities são classes, criadas pela API EF, onde:

- Cada *Entity* mapeia uma tabela da base de dados
- E as propriedades (*Scalar*) mapeiam as colunas da respetiva tabela;



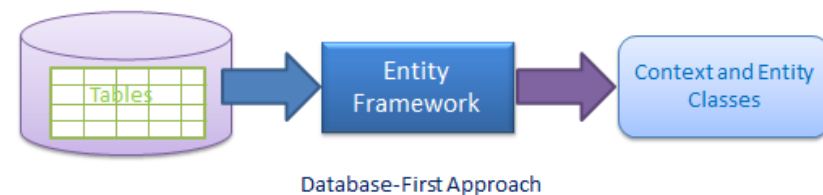
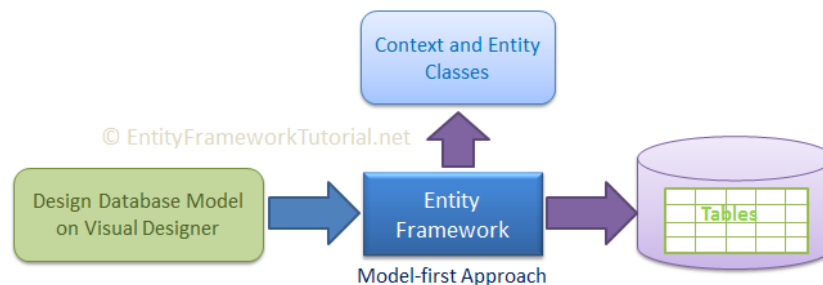
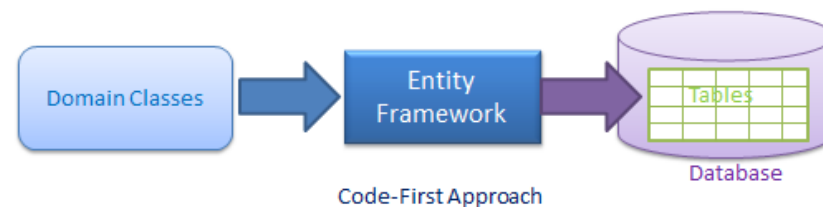
TIPOS DE ABORDAGENS

- **Código primeiro (Code First)** – gerar a base de dados ou/e o modelo a partir de código previamente escrito pelo programador
- **Modelo Primeiro (Model First)** – gerar a base de dados a partir de um modelo previamente definido pelo programador
- **Base de Dados Primeiro (Database First)** – gerar o modelo a partir de uma base de dados existente



TIPOS DE ABORDAGENS

- **Código primeiro (Code First)** – gerar a base de dados ou/e o modelo a partir de código previamente escrito pelo programador
- ~~**Modelo Primeiro (Model First)** – gerar a base de dados a partir de um modelo previamente definido pelo programador~~
- ~~**Base de Dados Primeiro (Database First)** – gerar o modelo a partir de uma base de dados existente~~



PROCESSO CODE FIRST - EXEMPLO

Passos:

1. Criar a aplicação e modelo de dados (classes)
2. Criar o Context (DbContext)
3. Utilizar Context para leitura/escrita de dados
4. Efetuar alterações no modelo (se necessário)



CODE FIRST – MODELO DE DADOS

- O modelo será gerado pelas classes incluídas no Context
 - Cada classe irá dar lugar a uma tabela
-
- Relacionamento de Associação (**Association**)
 - Relacionamentos de Herança (**Inheritance**)

```
public class Cozinheiro
{
    public int Id { get; set; }
    public string Nome { get; set; }
    public virtual List<Prato> Pratos { get; set; }
}
```

```
public class Prato
{
    public int Id { get; set; }
    public string Nome { get; set; }
    public string Ingredientes { get; set; }
    public int Rating { get; set; }
}
```

CODE FIRST – CRIAR CONTEXT

- Criar uma nova class `ApplicationContext`
- Herdar a class `DbContext` (EntityFramework)
- Definir as classes a incluir na base de dados da seguinte forma:

```
public class ApplicationContext : DbContext
{
    public DbSet<Cozinheiro> Cozinheiros { get; set; }
    public DbSet<Prato> Pratos { get; set; }
}
```

CODE FIRST – UTILIZAR CONTEXT

(1)

Persistência de dados

- Escrever dados

```
using (var db = new ApplicationDbContext())
{
    var cozinheiro = new Cozinheiro { Nome = "O Cozinheiro" };
    db.Cozinheiros.Add(cozinheiro);
    db.SaveChanges();
}
```

- Aceder a dados

```
var listCozinheiros = db.Cozinheiros.ToList();
```

CODE FIRST – UTILIZAR CONTEXT (2)

Utilização do DbContext para evitar problemas de runtime

- Usar uma instância por operação/evento
- Não guardar instâncias em propriedades
- Converter dados em List antes de utilizar de maneira a evitar lazy loading ou inconsistências

```
using (var db = new ApplicationDbContext()) {  
    return db.Users.ToList()  
}
```

- Minimizar o uso de código EntityFramework dentro dos forms:
 - Usar controladores/services

CODE FIRST – ALTERAÇÕES AO MODELO (1)

Refazer automaticamente a BD

- No main (program.cs) colocar:

```
using System.Data.Entity;
```

```
Database.SetInitializer(  
    new DropCreateDatabaseIfModelChanges<AppDbContext>()  
);
```

- Cada vez que a app inicializa verifica se os modelos são diferentes e refaz a BD novamente
- **Perdemos** os dados inseridos mas...

CODE FIRST – ALTERAÇÕES AO MODELO (2)

BD seeding - Inicialização da base com dados pré-definidos

- Criar uma nova classe:

```
public class AppDbInitializer : DropCreateDatabaseIfModelChanges<AppDbContext>
{
    protected override void Seed(AppDbContext context)
    {
        // Alterar para o vosso context
        context.Customers.Add(new Customer { Name = "Test User" });
        base.Seed(context);
    }
}
```

- No program.cs iniciar com:

```
Database.SetInitializer(new AppDbInitializer());
```

FONTES

- ✓ <http://www.entityframeworktutorial.net/>
- ✓ https://www.tutorialspoint.com/entity_framework/index.htm